

第八章

8.1 使用 Camera2 实现定时拍照的功能。

```
private static final int CAMERA_ID = 0; //后置摄像头
// private static final int CAMERA_ID = 1; //前置摄像头
private static final String TAG = MainActivity.class.getSimpleName();

@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);

    initView();
    initEvent();
}

@Override
protected void onPause() {
    super.onPause();

    stopPreview();
}

private void initView() {
    mSurfaceView = (SurfaceView) findViewById(R.id.surface_view);
    mIvStart = (ImageView) findViewById(R.id.start);
    mTvCountDown = (TextView) findViewById(R.id.count_down);
}

private void initEvent() {
    mHolder = mSurfaceView.getHolder();
    mHolder.addCallback(this);

    mIvStart.setOnClickListener(this);
}

@Override
public void surfaceCreated(SurfaceHolder holder) {
    mIsSurfaceCreated = true;
}

@Override
public void surfaceChanged(SurfaceHolder holder, int format, int width, int height)
```

```

{
    startPreview();
}
@Override
public void surfaceDestroyed(SurfaceHolder holder) {
    mIsSurfaceCreated = false;
}
private void startPreview() {
    if (mCamera != null || !mIsSurfaceCreated) {
        Log.d(TAG, "startPreview will return");
        return;
    }
    mCamera = Camera.open(CAMERA_ID);
    Camera.Parameters parameters = mCamera.getParameters();
    int width = getResources().getDisplayMetrics().widthPixels;
    int height = getResources().getDisplayMetrics().heightPixels;
    Camera.Size size = getBestPreviewSize(width, height, parameters);
    if (size != null) {
        //设置预览分辨率
        parameters.setPreviewSize(size.width, size.height);
        //设置保存图片的大小
        parameters.setPictureSize(size.width, size.height);
    }

    //自动对焦
    parameters.setFocusMode(Camera.Parameters.FOCUS_MODE_AUTO);
    parameters.setPreviewFrameRate(20);

    //设置相机预览方向
    mCamera.setDisplayOrientation(90);

    mCamera.setParameters(parameters);

    try {
        mCamera.setPreviewDisplay(mHolder);
    } catch (Exception e) {
        Log.d(TAG, e.getMessage());
    }

    mCamera.startPreview();
}

private void stopPreview() {
    //释放 Camera 对象

```

```

        if (mCamera != null) {
            try {
                mCamera.setPreviewDisplay(null);
                mCamera.stopPreview();
                mCamera.release();
                mCamera = null;
            } catch (Exception e) {
                Log.e(TAG, e.getMessage());
            }
        }
    }

    private Camera.Size getBestPreviewSize(int width, int height,
                                           Camera.Parameters parameters)
    {
        Camera.Size result = null;

        for (Camera.Size size : parameters.getSupportedPreviewSizes()) {
            if (size.width <= width && size.height <= height) {
                if (result == null) {
                    result = size;
                } else {
                    int resultArea = result.width * result.height;
                    int newArea = size.width * size.height;

                    if (newArea > resultArea) {
                        result = size;
                    }
                }
            }
        }

        return result;
    }

    @Override
    public void onClick(View v) {
        switch (v.getId()) {
            case R.id.start:
                if (!mIsTimerRunning) {
                    mIsTimerRunning = true;
                    mHandler.post(timerRunnable);
                }
                break;
        }
    }
}

```

```

private Runnable timerRunnable = new Runnable() {
    @Override
    public void run() {
        if (mCurrentTimer > 0) {
            mTvCountDown.setText(mCurrentTimer + "");

            mCurrentTimer--;
            mHandler.postDelayed(timerRunnable, 1000);
        } else {
            mTvCountDown.setText("");

            mCamera.takePicture(null, null, null, MainActivity.this);
            playSound();

            mIsTimerRunning = false;
            mCurrentTimer = 10;
        }
    }
};

```

```

@Override
public void onPictureTaken(byte[] data, Camera camera) {
    try {
        FileOutputStream fos = new FileOutputStream(new File
            (Environment.getExternalStorageDirectory() + File.separator
+
                System.currentTimeMillis() + ".png"));

        //旋转角度，保证保存的图片方向是对的
        Bitmap bitmap = BitmapFactory.decodeByteArray(data, 0, data.length);
        Matrix matrix = new Matrix();
        matrix.setRotate(90);
        bitmap = Bitmap.createBitmap(bitmap, 0, 0,
            bitmap.getWidth(), bitmap.getHeight(), matrix, true);
        bitmap.compress(Bitmap.CompressFormat.PNG, 100, fos);
        fos.flush();
        fos.close();
    } catch (FileNotFoundException e) {
        e.printStackTrace();
    } catch (IOException e) {
        e.printStackTrace();
    }
}

```

```

        mCamera.startPreview();
    }
    /**
     * 播放系统拍照声音
     */
    public void playSound() {
        MediaPlayer mediaPlayer = null;
        AudioManager audioManager = (AudioManager)
getSystemService(Context.AUDIO_SERVICE);
        int volume =
audioManager.getStreamVolume( AudioManager.STREAM_NOTIFICATION);

        if (volume != 0) {
            if (mediaPlayer == null)
                mediaPlayer = MediaPlayer.create(this,

Uri.parse("file:///system/media/audio/ui/camera_click.ogg"));
            if (mediaPlayer != null) {
                mediaPlayer.start();
            }
        }
    }
}

```

